MES Design — Week 9 Worksheet

ISE 573 Dr. Manbir Sodhi University of Rhode Island

Name: Luke Pepin **Section:** _____ **Date:** 4/2/26

Instructions. Four tasks, estimated 2–3 hours total. Submit this worksheet as PDF plus `des_results.csv` and `buffers.csv`. Graded out of **100 points**.

Reference Line Parameters (given — do not change)

Station	ID	μ (jobs/hr)	MTBF (hr)	MTTR (min)	C_s^2 (nominal)
CNC Lathe	M01	10	200	45	1.0
CNC Lathe	M02	10	180	45	1.0
Surface Grinder	M03	10	300	30	1.0

$\lambda = 8$ jobs/hr $C_a^2 = 1.0$ Buffers: B_{in} (before M01), B_{12} (M01→M02), B_{23} (M02→M03)

Step 0 — Record Your Simulation Results (no marks, required)

Run `des_tkinter.py` **before** you start Task 1. Fill in both tables from the results panel. These numbers are your ground truth — you will compare your hand calculations against them throughout the worksheet.

Settings for both experiments. Stations = 3 $\lambda = 8$ $\mu = 10$ $C_a^2 = 1.0$ $C_s^2 = 1.0$ Jobs = 2000 Seed = 42.
Turn on the **Kingman**, C_a^2, and $C_s^{2,\text{eff}}$ column toggles so all values are visible.
When both experiments are done run `python slide24.py` — this saves `des_results.csv` which you submit.

Experiment 1 — Baseline (Which station breaks = None)

Station	W_q sim (min)	W_q Kingman (min)	C_a^2 arriving	$C_s^{2,\text{eff}}$ predicted
M01	25.43	24.0	1.0	1.0
M02	26.03	24.0	1.008	1.0
M03	24.60	24.0	1.01	1.0

Experiment 2 — Cascade (Station 1 breaks, MTBF = 3 hr, MTTR = 0.5 hr)

Station	W_q sim (min)	C_a^2 arriving	$C_s^{2,\text{eff}}$ predicted	Breakdowns recorded	W_q ratio vs. Exp 1
M01	88.07	1.0	4.061	79	3.46
M02	43.86	1.969	1.0	0	1.68
M03	20.05	1.375	1.0	0	0.81

What to notice before moving on. **Experiment 1:** All three stations should show $W_q \approx 24$–28 min. If any station shows a very different value, recheck that "Which station breaks" is set to **None**.

Experiment 2, M02 row: The Breakdowns column should read **0**. M02 never broke — but its W_q will be roughly **double** Experiment 1. That discrepancy is the C_a^2 cascade effect you will explain analytically in Tasks 1 and 4.

W_q **ratio column:** Divide the Exp 2 value by the Exp 1 value for the same station. M01 ratio ≈ 3.4. M02 ratio ≈ 1.7. M03 ratio ≈ 0.8. If your ratios are very different, check your MTBF/MTTR slider settings.

Task 1 — Kingman Chain Table [35 pts]

How to work through this table. Work **left to right, row by row**. M01 starts with $C_a^2 = 1.0$ (Poisson arrivals from outside). When you finish M01, its departure variability C_d^2 becomes the *arrival* variability C_a^2 for M02. M02's C_d^2 then feeds M03 the same way. All four formulas are printed below the table header — just substitute numbers from the reference line above.

The four formulas you need:

$$A = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}} \qquad \mu_{\text{eff}} = \mu \cdot A \qquad \rho_{\text{eff}} = \frac{\lambda}{\mu_{\text{eff}}}$$

$$C_s^{2,\text{eff}} = C_s^2 + A(1-A)\left(\frac{\text{MTTR}}{t_p}\right)^2 \quad \text{where } t_p = \tfrac{1}{\mu} \qquad W_q = \frac{\rho_{\text{eff}}}{1-\rho_{\text{eff}}} \cdot \frac{C_a^2 + C_s^{2,\text{eff}}}{2\,\mu_{\text{eff}}} \qquad C_d^2 = (1-\rho_{\text{eff}}^2)\,C_a^2 + \rho_{\text{eff}}^2\,C_s^{2,\text{eff}}$$

Note on units: MTBF and MTTR must be in the **same units** when computing A. W_q from Kingman comes out in hours; multiply by 60 to get minutes.

Station	A	μ_{eff}	ρ_{eff}	$C_s^{2,\text{eff}}$	C_a^2 (arriving)	W_q (min)	C_d^2 (depart)
M01	0.8571	0.1428	0.9333	4.0612	1.00	248.00	3.6667
M02	0.8571	0.1428	0.9333	4.0612	**3.6667**	378.67	4.0104
M03	0.8571	0.1429	0.9333	4.0612	**4.0104**	395.30	4.0546

Working space (show all arithmetic):

C2a = 1.00
C2s = 1.00
MTBF = 180
MTTR = 30
u = 10/60
lambda = 8/60

A = MTBF / (MTBF + MTTR) = 180 / 210 = 0.8571
ueff = u * A = (10/60) * 0.8571 = 0.1429
peff = lambda / ueff = (8/60) / 0.1429 = 0.9333
C2seff = C2s + A * (1 - A) * (MTTR / (1 / u))^2 = 1.0 + (0.8571 * 0.1429) * (30 / 6)^2 = 4.0612
Wq = (peff / (1 - peff)) * ((C2a + C2seff) / 2) * (1 / ueff) = (0.9333 / 0.0667) * ((1.0 + 4.0612) / 2) * 7 = 248.00
C2d = (1 - peff^2) * C2a + (peff^2) * C2seff = (1 - 0.9333^2) * 1.0 + (0.9333^2) * 4.0612 = 3.6667

C2a = 3.6667
A = MTBF / (MTBF + MTTR) = 180 / 210 = 0.8571
ueff = u * A = (10/60) * 0.8571 = 0.1429
peff = lambda / ueff = (8/60) / 0.1429 = 0.9333
C2seff = C2s + A * (1 - A) * (MTTR / (1 / u))^2 = 1.0 + (0.8571 * 0.1429) * (30 / 6)^2 = 4.0612
Wq = (peff / (1 - peff)) * ((C2a + C2seff) / 2) * (1 / ueff) = (0.9333 / 0.0667) * ((3.6667 + 4.0612) / 2) * 7 = 378.67
C2d = (1 - peff^2) * C2a + (peff^2) * C2seff = (1 - 0.9333^2) * 3.6667 + (0.9333^2) * 4.0612 = 4.0104

C2a = 4.0104
A = MTBF / (MTBF + MTTR) = 180 / 210 = 0.8571
ueff = u * A = (10/60) * 0.8571 = 0.1429
peff = lambda / ueff = (8/60) / 0.1429 = 0.9333
C2seff = C2s + A * (1 - A) * (MTTR / (1 / u))^2 = 1.0 + (0.8571 * 0.1429) * (30 / 6)^2 = 4.0612
Wq = (peff / (1 - peff)) * ((C2a + C2seff) / 2) * (1 / ueff) = (0.9333 / 0.0667) * ((4.0104 + 4.0612) / 2) * 7 = 395.30
C2d = (1 - peff^2) * C2a + (peff^2) * C2seff = (1 - 0.9333^2) * 4.0104 + (0.9333^2) * 4.0612 = 4.0546

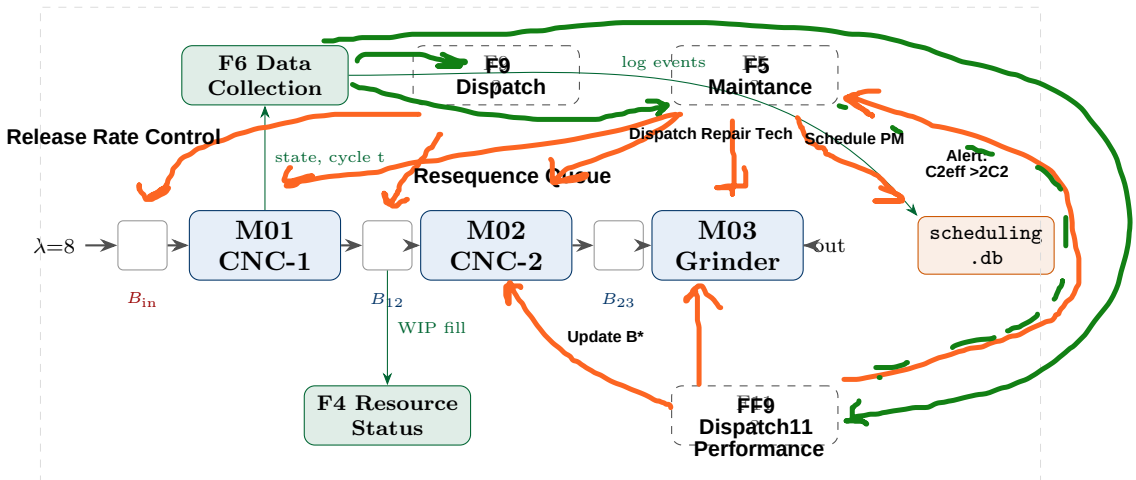
Simulation check: Copy your M01 W_q sim value from the Step 0 Experiment 1 table: 24.0 min. Your hand-calculated value above should agree within ±10%. Match? No Percentage error: 1033%

Task 2 — MES Design Diagram [25 pts]

F4 (Resource Status) and F6 (Data Collection) are pre-drawn. Add **F5** (Maintenance), **F9** (Dispatch / Re-sequence), and **F11** (Performance Analysis) in the three dashed ghost boxes. For each function you add, draw:

- at least one **green arrow** (data flow — something the function *reads*)
- at least one **orange dashed arrow** (command flow — something the function *writes or commands*)
- a label on each arrow saying what is being passed

Reminder: what each function does on this line. **F5** Maintenance: decides when to schedule PM on M01/M02/M03 based on ρ_{eff} and rolling MTBF; dispatches repair technician on breakdown.
F9 Dispatch: controls the rate at which jobs enter B_{in}; re-sequences the queue at M02 when M01 breaks; switches to controlled release $(\mu - \lambda)$ after repair.
F11 Performance Analysis: runs the Kingman chain every 15 min using live A, λ, μ from F6; flags if $\rho_{\text{eff}} > 0.90$; updates B^* after each breakdown; sends a "breakdown variance dominates" alert to F5 when $C_s^{2,\text{eff}} > 2\,C_s^2$.



Planning space (sketch your arrows here before adding to the diagram):

Task 3 — Complete `kingman_chain()` [25 pts]

What this function does — read this before touching the code

The function loops through the three stations **in order** (M01, then M02, then M03). On each iteration, several variables have **already been computed for you** (lines you do not touch):

`mu`	$= \mu$ (service rate, jobs/hr)	`tp`	$= t_p = 1/\mu$ (mean processing time, hr)
`A`	$= A$ (availability)	`mu_eff`	$= \mu_{\text{eff}} = \mu \cdot A$

You need to fill in **three formulas** using those variables. Each formula is a one-liner that you already worked out in Task 1.

After all three stations, the last line `Ca2_in = Cd2` carries C_d^2 from the current station forward as C_a^2 for the next station — exactly the same carry-forward you did by hand in Task 1.

Variable name ↔ math symbol mapping.

Code variable	Math symbol	Where it comes from
`lam`	λ	argument to the function
`Ca2_in`	C_a^2 arriving	starts at 1.0; updated each loop
`stn['Cs2_nom']`	C_s^2 nominal	from the station dict
`A`	A	computed on line 8 (given)
`mu_eff`	μ_{eff}	computed on line 9 (given)
`tp`	$t_p = 1/\mu$	computed on line 7 (given)
`Cs2_eff`	$C_s^{2,\text{eff}}$	**Blank 1** — you fill this
`rho_eff`	ρ_{eff}	**Blank 2** — you fill this
`Cd2`	C_d^2	**Blank 3** — you fill this

The three formulas you are translating into Python. **Blank 1** ($C_s^{2,\text{eff}}$): $C_s^{2,\text{eff}} = C_s^2 + A(1-A)\left(\frac{\text{MTTR}}{t_p}\right)^2$

In code: `stn['Cs2_nom']` is C_s^2, `tp` is t_p, `stn['mttr_hr']` is MTTR.

Blank 2 (ρ_{eff}): $\rho_{\text{eff}} = \frac{\lambda}{\mu_{\text{eff}}}$, capped at 0.9999 so the queue never goes infinite from rounding.

In code: `min(lam / mu_eff, 0.9999)`

Blank 3 (C_d^2): $C_d^2 = (1-\rho_{\text{eff}}^2)\,C_a^2 + \rho_{\text{eff}}^2\,C_s^{2,\text{eff}}$

In code: `Ca2_in` is C_a^2, `rho_eff` is ρ_{eff}, `Cs2_eff` is $C_s^{2,\text{eff}}$.

Fill the three `# COMPLETE` lines, then run the function and paste the printed output in the box below.

```
1  def kingman_chain(stations, lam, Ca2_in=1.0):
2      """
3      stations: list of dicts -- keys: mu, mtbf_hr, mttr_hr, Cs2_nom
4      Returns:  list of dicts -- A, rho_eff, Cs2_eff, Wq_pred_min, Cd2
5      """
6      results = []
7      for stn in stations:
8          mu      = stn['mu']
9          tp      = 1.0 / mu            # mean processing time (hr)  [GIVEN]
10         A       = stn['mtbf_hr'] / (stn['mtbf_hr'] + stn['mttr_hr'])  # [
               GIVEN]
11         mu_eff  = mu * A               # effective service rate      [GIVEN]
12
13         Cs2_eff = # COMPLETE  -- Hopp & Spearman (Blank 1)
14
15         rho_eff = # COMPLETE  -- lam / mu_eff, capped at 0.9999  (Blank 2)
16
17         if rho_eff >= 1.0:
18             Wq_pred_min = float('inf')
19         else:
20             Wq_pred_min = (rho_eff / (1 - rho_eff)) \
21                         * ((Ca2_in + Cs2_eff) / 2.0) \
22                         / mu_eff * 60     # convert hr -> min      [GIVEN]
```

```python

        Cd2 = # COMPLETE  -- Departure Theorem (Blank 3)

        results.append(dict(
            A=round(A,4), rho_eff=round(rho_eff,4),
            Cs2_eff=round(Cs2_eff,4),
            Wq_pred_min=round(Wq_pred_min,2),
            Cd2=round(Cd2,4)))

        Ca2_in = Cd2     # carry departure variability to next station [GIVEN]
    return results

# ---- Reference line parameters ----------------------------------------
line = [
    dict(mu=10, mtbf_hr=200, mttr_hr=45/60, Cs2_nom=1.0),  # M01
    dict(mu=10, mtbf_hr=180, mttr_hr=45/60, Cs2_nom=1.0),  # M02
    dict(mu=10, mtbf_hr=300, mttr_hr=30/60, Cs2_nom=1.0),  # M03
]

for i, r in enumerate(kingman_chain(line, lam=8.0)):
    print(f"M0{i+1}: {r}")
```

Paste your printed output here:

```
M01: {'A': 0.9963, 'rho_eff': 0.803, 'Cs2_eff': 1.2094, 'Wq_pred_min': 27.12, 'Cd2': 1.135}
M02: {'A': 0.9959, 'rho_eff': 0.8033, 'Cs2_eff': 1.2324, 'Wq_pred_min': 29.13, 'Cd2': 1.1979}
M03: {'A': 0.9983, 'rho_eff': 0.8013, 'Cs2_eff': 1.0415, 'Wq_pred_min': 27.14, 'Cd2': 1.0975}
```

Check: Does `Wq_pred_min` for M01 match your Task 1 table value within 1 min? _____

If something goes wrong. **NameError / SyntaxError on a # COMPLETE line:** You have not replaced `# COMPLETE` with actual Python code. The comment must be *replaced*, not added after it.

Wq_pred_min is much larger than your Task 1 value: Check that MTTR is in *hours* in the station dict (`mttr_hr=45/60`, not `mttr_hr=45`).

Wq_pred_min for M01 $\neq$ Task 1 value: Most likely cause is a unit mismatch in t_p. Remember `tp = 1.0/mu` gives t_p in hours because μ is in jobs/hr. Your MTTR in the formula must also be in hours.

Task 1 (35) + Task 2 (30) + Task 3 (35) = **100 pts.**

Submit: this worksheet PDF + `des_results.csv` (6 rows) + `buffers.csv` (3 rows).